DELOITTE CYBER GURUKUL & NLA MASTER COMPENDIUM

Complete Collection of 22 Core Coding Questions & High-Quality Python 3 Solutions

SECTION 1: PLATFORM-SPECIFIC SCENARIO QUESTIONS

1. Corona for Computer (Bitwise Shift)

Problem: A computer virus strips away the n least significant bits (LSB) from each processing token in a data sequence array.

Complexity: Time: O(N) | Space: O(1)

```
N = int(input())
arr = list(map(int, input().split()))
n = int(input())
print(*(x >> n for x in arr))
```

2. Particle Momentum

Problem: Read the mass and velocity of n particles and compute the total system momentum (Σ m × v).

```
n = int(input())
total_momentum = sum(m * v for _ in range(n) for m, v in [map(int,
input().split())])
print(total_momentum)
```

3. Odd Sum Subarrays

Problem: Count the number of continuous contiguous subarrays that sum up to an odd integer.

```
N = int(input())
arr = list(map(int, input().split()))
odd_c, even_c, res, pref = 0, 1, 0, 0
for num in arr:
    pref += num
    if pref % 2 == 0:
        res += odd_c
        even_c += 1
    else:
        res += even_c
        odd_c += 1
print(res)
```

4. Class Monitor (Rank Drop Count)

Problem: Count how many times values in an array fall strictly below the running minimum recorded up to that index.

```
n = int(input())
scores = list(map(int, input().split()))
if not scores:
    print(0)
else:
    curr_min, drops = scores[0], 0
    for s in scores[1:]:
        if s < curr_min:
            curr_min, drops = s, drops + 1
    print(drops)
```

5. Key Generation (Alphanumeric Digits Extraction)

Problem: Parse a string token, extract all numeric characters, and print the sum of their individual base-10 digits.

```
s = input()
print(sum(int(char) for char in s if char.isdigit()))
```

6. Prime Sum with Ending Digit 'K'

Problem: Extract and compute the cumulative sum of the first N prime numbers that explicitly terminate with the base digit K.

```
def is_prime(num):
    if num < 2: return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0: return False
    return True

N, K = map(int, input().split())
primes_found, current_num, total_sum = 0, 2, 0
while primes_found < N:
    if current_num % 10 == K and is_prime(current_num):
        total_sum += current_num
        primes_found += 1
    current_num += 1
print(total_sum)
```

7. Reference String Filter

Problem: Clean a target string payload by stripping out any characters present anywhere within a designated reference filter string.

```
target = input()
reference = set(input())
print("".join([char for char in target if char not in reference]))
```

8. Dynamic String Space-Pairing

Problem: Reformat an input sequence token string by systematically embedding an explicit whitespace character after every 2 characters.

```
s = input().replace(" ", "")
print(" ".join(s[i:i+2] for i in range(0, len(s), 2)))
```

SECTION 2: CORE DATA STRUCTURES & OPTIMIZATION POOL

9. Maximum Product Subarray

Problem: Find the contiguous subsegment within a dynamic integer array stream that produces the maximum product.

```
n = int(input())
arr = list(map(int, input().split()))
if not arr:
    print(0)
else:
    max_prod = min_prod = ans = arr[0]
    for x in arr[1:]:
        if x < 0: max_prod, min_prod = min_prod, max_prod
        max_prod = max(x, max_prod * x)
        min_prod = min(x, min_prod * x)
        ans = max(ans, max_prod)
    print(ans)
```

10. Distinct Elements After Sign Modification

Problem: Maximize the count of completely distinct items in an array if you can toggle any item's sign (positive/negative) at most once.

```
n = int(input())
arr = list(map(int, input().split()))
seen = set()
for x in arr:
    abs_x = abs(x)
    if abs_x not in seen: seen.add(abs_x)
    else: seen.add(-abs_x)
print(len(seen))
```

11. Encryption Key Redundancy (Adjacent Reductions)

Problem: Minimize a string payload by checking and dropping duplicate adjacent matching character pairs recursively.

```
s = input()
stack = []
for char in s:
    if stack and stack[-1] == char: stack.pop()
    else: stack.append(char)
ans = "".join(stack)
print(ans if ans else "Empty String")
```

12. Unusual Token Pairs (Absolute Difference K)

Problem: Count unique pairs of tracking elements (A, B) whose exact absolute arithmetic difference matches the constraint value K.

```
from collections import Counter
n = int(input())
arr = list(map(int, input().split()))
k = int(input())
seen = set(arr)
count = 0
if k == 0:
    counts = Counter(arr)
    count = sum(1 for x in counts if counts[x] > 1)
else:
    for x in seen:
        if x + k in seen: count += 1
print(count)
```

13. Server Load Balancing (Equilibrium Balance Index)

Problem: Find the exact structural index position where the weight sum of all left nodes matches the weight sum of all right nodes.

```
n = int(input())
weights = list(map(int, input().split()))
total_sum, left_sum, eq_idx = sum(weights), 0, -1
for i, w in enumerate(weights):
    total_sum -= w
    if left_sum == total_sum:
        eq_idx = i
        break
    left_sum += w
print(eq_idx)
```

14. Bitwise Transmission Parity

Problem: Identify the unique numerical data packet that is broadcast an odd number of times when all others appear an even number of times.

```
n = int(input())
packets = list(map(int, input().split()))
res = 0
for p in packets: res ^= p
print(res)
```

SECTION 3: RECENT HIGH-FREQUENCY NLA RECYCLED ADDITIONS

15. Chocolate Factory Logic (Push Zeros to End)

Problem: Relocate all zero data values to the rear boundary of the array stream without skewing the sequence order of valid integers.

```
n = int(input())
arr = [int(input()) for _ in range(n)]
non_zeros = [x for x in arr if x != 0]
print(*(non_zeros + [0] * (n - len(non_zeros))))
```

16. Character-to-Binary Payload Masking

Problem: Flag string vowels as 0 and consonants as 1, concatenate into a binary representation, and output the base-10 integer translation.

```
s = input()
vowels = set("aeiou")
bin_str = "".join("0" if c in vowels else "1" for c in s)
print(int(bin_str, 2))
```

17. Complement Bit Toggling

Problem: Calculate the full structural bitwise complement inversion of a value beginning explicitly at its Most Significant Bit (MSB).

```
n = int(input())
mask = (1 << n.bit_length()) - 1
print(n ^ mask)
```

18. Binary String Stream Addition

Problem: Add two space-separated string tokens representing large binary streams and display the aggregate value as a raw binary string.

```
a, b = input().split()
print(bin(int(a, 2) + int(b, 2))[2:])
```

19. Multi-Element Threshold Bounds Counter

Problem: Return the total frequency count of elements that satisfy the structural variance bound rule: |arr[i] - num| ≤ diff.

```
length = int(input())
arr = [int(input()) for _ in range(length)]
num = int(input())
diff = int(input())
count = sum(1 for x in arr if abs(x - num) <= diff)
print(count if count > 0 else -1)
```

20. Divisibility Difference Matrix Accumulator

Problem: Find the absolute difference between numbers up to m that are cleanly divisible by n versus those that are not.

```
n = int(input())
m = int(input())
div_sum = sum(i for i in range(1, m + 1) if i % n == 0)
not_div_sum = sum(i for i in range(1, m + 1) if i % n != 0)
print(abs(div_sum - not_div_sum))
```

21. Core String Alternator (Character Interleaving)

Problem: Iteratively interleave characters from two string inputs. Directly append any residual suffix text segments remaining from the longer input string.

```
s1, s2 = input().split()
res, min_len = [], min(len(s1), len(s2))
for i in range(min_len): res.append(s1[i] + s2[i])
res.append(s1[min_len:] + s2[min_len:])
print("".join(res))
```

22. Risk Severity Multi-Pointer Sort (0, 1, 2 Dutch Flag Variant)

Problem: Unpack and sort an array consisting strictly of risk metrics 0, 1, and 2 in a strict single-pass O(N) configuration without utilizing sort functions.

```
n = int(input())
arr = [int(input()) for _ in range(n)]
low, mid, high = 0, 0, n - 1
while mid <= high:
    if arr[mid] == 0:
        arr[low], arr[mid] = arr[mid], arr[low]
        low, mid = low + 1, mid + 1
    elif arr[mid] == 1:
        mid += 1
    else:
        arr[mid], arr[high] = arr[high], arr[mid]
        high -= 1
print(*arr)
```

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.